Development Plan
Software Engineering

Team #14, Troitsky Toolbox
Pedro Azevedo
Mason Azzopardi
Michael Roberts
Jack Easton

Table 1: Revision History

Date	**Developer(s)**	**Change**
09/23/2026	Jack, Mason, Pedro, and Michael	First draft of document created.
09/27/2026	Pedro	Team Charter finialization.
09/28/2026	Jack, Mason, Pedro, and Michael	Completed version 1 of document.

1 Introduction

This document will outline the general development plan of the Troitsky Toolbox project. Sections 1-3 outline the project information and protection information; sections 4-8 describe team member roles, workflow plan, project decomposition and scheduling; sections 9-12 cover technical components of the development process, such as the final proof-of-concept demonstration plan, technology used, and AI use.

2 Confidential Information?

This project does not depend on confidential information. However, once deployed, some documents held in the knowledge repository will be confidential and will be protected through industry-standard measures. All public demonstrations shall exclude these documents, using dummy documents in their stead.

3 IP to Protect

There is no IP to protect in this project.

4 Copyright License

Our Copyright License reserves all rights. No permission is granted to use, copy, modify, or distribute this code without prior written permission.

5 Team Meeting Plan

5.1 Team Meet

Weekly Meetings will occur every Wednesday at 1:30 PM. These Meetings will either be in person or online, meetings are mandatory in person if they occur within a week of a deliverable worth 10% of the total course grade.

5.2 Supervisor Meeting

Meeting with our supervisor will occur once every other Wednesday during the Troitsky executive meeting.

5.3 Meeting Structure

5.3.1 Roles

Roles will be taken by the most relavent individual assigned to them for each meeting

- **Chair (Mason/Michael)**: The meeting lead; takes charge of organizing the group and ensuring the meeting is on track to cover and complete the predesignated meeting topics.
- **Notery (Pedro/Jack)**: The note taker; makes records of each topic discussed in a meeting summarizing the issues covered and plans to address them.

5.3.2 Agenda Structure

The Agenda will follow the below structure:

Purpose of Meeting
Describe reason for meeting

Date and Time
XX:XX DD/MM/YYYY

Attendance
□Jack
□Mason
□Pedro
□Micheal

Items To Address
List items that are relevent to the meeting and the plan to tackle them

---

6 Team Communication Plan

The Team Communication Plan outlines what means of communication will be used for what purpose.

6.1 GitHub

GitHub will act as our main file sharing platform, both LaTeX documents for submission like this development plan and the code used in the development of the Toolbox will be stored and edited through GitHub. Additionally, all work assignments will be outlined as issues on the KanBan board along with the Meeting Agendas, Peer Reviews, and Lecture Overviews.

6.2 Teams

Microsoft Teams will act as our online meeting platform and a secondary file sharing platform, for files too large to push to GitHub or that would cause clutter inside of the GitHub.

6.3 Text Message Group Chat

The Text Message Group Chat will be used for simple updates/questions on how work is coming along, for example asking when a group member will be done reviewing an issue or for quick meeting reminders.

7 Team Member Roles

7.1 Role Descriptions

1. **Manager:** The manager will serve as a coordinator between the other members, ensuring that communication is constant and issues are resolved in time and amicably. The manager will also serve as the standard meeting lead, with room for this to change as meeting topics become specified.

2. **Notetaker:** The notetaker will be responsible for taking meeting minutes in supervisor, TA and general team meetings, along with taking notes during lectures for the team. This role will focus on maintaining a standardized note structure and level of detail throughout sessions.

3. **Troitsky Club Communication Associate:** This role will be responsible for maintaining constant communication with the Troitsky club, scheduling interviews, user tests, and any other interaction the team will have with the club. This will become a more generalized role as the team becomes more integrated into the club.

4. **Reviewer/Administrator:** The reviewer/administrator role will be responsible for final reviews of PRs and ensuring that submissions are complete by the set deadlines. While the team as a whole should be integrated into this process, the administrator will serve as the final check and insurance that items will be complete on time.

7.2 Role Assignments

The assignment structure of the team will be chosen initially based on prior experience and general inclination to each role. The reassignment of these roles will be completed between submissions, focusing on allowing each member to take on the given roles. In cases of changes and new information arising, the role assignments and structure may be adjusted through communication with all members.

8 Workflow Plan

The workflow process used during the development of Troitsky's Toolbox will be centred around GitHub, and done in sprints which will begin and end at every major milestone. Issues within our KanBan will be the main feature we use to track and assign work.

8.1 Workflow Overview

1. At the beginning of a sprint all major tasks will be broken down into issues and assigned to group members based on their specialties and interests.
2. New branches will be made for each issue from the most recent version of the main branch, the issues will be related to their respective branches via the development feature in the issue.
3. If multiple team members are assign to the same issue they will create sub-issues under said issue and create new branches from the parent issues branch.
4. When sub-issues are completed they will be merged to the parent branch with a Pull Request (PR), and reviewed by a team member working under the same parent issue.
5. When issues are completed they will be merged through a PR to main and review by a group member that did not work on said issue, if possible.
6. When a PR is made it will be run through the CI/CD pipeline using GitHub actions, and can only be reviewed and merged if it passes all tests.
7. Once a branch has been reviewed, merged, and passed the CI/CD pipeline it will be deleted.

8.2 Issue Management

At the beginning of each sprint, issues and their member assignments will be outlined in our weekly meeting and added to the KanBan as issues by the Current Administrator. It is up to those working on issues to outline their own sub-issues. Issues will be classified via the tools they relate to and if they are backend or frontend as tags (i.e, 'Rule Compliance Tester', 'Frontend'). All fixes to issues shall occur on development branches, be tested and determined to be correct by the developer before submitting a pull request. Then, another developer shall review the code changes and must approve it before the code may enter production. If there is a merge conflict, the onus is on the original developer to resolve the merge conflict, determine the altered codes satisifies tests, and resubmit the code for peer review.

- How will you be using git, including branches, pull request, etc.?
- How will you be managing issues, including template issues, issue classification, etc.?
- Use of CI/CD

9 Project Decomposition and Scheduling

We will be using our GitHub KanBan to outline issues and create a roadmap of major project dates.

Table 2: A tentative list of major project dates based on the course outline

Submission Items	**Date**
• Problem Statement • Proof of Concept Plan • Development Plan	28/09/2026
• Problem Statement Peer Review • Proof of Concept Plan Peer Review • Development Plan Peer Review	01/10/2026
• SRS • Hazard Analysis	19/10/2026
SRS Peer Review	22/10/2026
• VnV Peer Review Requests • POC Requests	05/11/2026
• Design Doc Rev 1 • Fall Custom Deliverable	19/11/2026
Module Implementation	21/01/2027
Exposition Registration	27/01/2027
Final Demo Schedule Requests	01/03/2027
• VnV • Custom Deliverable Winter	11/03/2027
Team Contribution Survey	18/03/2027
Final Submission	31/03/2027
Expo	06/04/2027

10 Proof of Concept Demonstration Plan

Due to the nature of the Toolbox providing a variety of services and features, the plan for the Proof of Concept Demonstration has a unique way it approaches the risks presented in the project.

10.1 Risks

Since many tools operate independently from each other, or in small groups, the risks presented by a single tool's performace/attainabliity do not directly impact the attainability of another.

With that being said, there is a group of tools and features that have a high level of risk associated with them due to their complexity and connectivity to each other. These related and complex tools would be uniquely tailored to assist the McMaster Troitsky with visualization, performance estimation, and rule compliance testing of bridge designs.

Since the specific details of rule compliance and performance estimation aren't relevant for the Proof of Concept Demo, the focus will be on extracting and visualizing data from the 3D models of a given bridge design. The plan to demonstrate the concept of functionality is possible by loading a STEP file (.stp – universal 3D CAD file format) and displaying a visualization or rendering of the file's contents. Along with the visualization, numerical data will be extracted from the .stp file related to the bridge, which eventually will be used to support the competition rule compliance assessment tool and projected bridge performance and metrics tool. This is the individually most complicated feature that will be implemented in the Troitsky Toolbox, so it serves as one of the greatest risks to the project.

It is also worth noting that although there are many different tools that will need to be appropriately connected within the program, it is not a large risk or feasible to demonstrate during the Proof of Concept Demo.

10.2 Risk Mitigation

Due to the nature of the Toolbox, risk mitigation in the specific project context is also unique. Since the biggest isloated risk comes from an individual tool, the biggest negative impact of that risk (if not successfully demonstrated) is that the Toolbox cannot provide that specific feature. For this project, a "failed" POC demo would mean that there are a few features (compared to the many planned to implement) that aren't achievable within the scope of the project. This failure means that the team would modify the scope of the project to include an alternate related feature that bypasses the technical limitation or refine, improve, iterate, and add to alternate functionality of the Toolbox.

11 Expected Technology

This project shall be accessible via browser for desktop users. Data shall be stored in a robust database. The back-end logic shall be rewritten a powerful and easily maintainable programming language, preferably with a utility which will aid front-end development. The current likely option is the .NET programming suite, backed by a Microsoft SQL Server database, with the back-end in C# and the front-end utilizing Razor (C# HTML) pages to facilitate rendering web pages. A great advantage of doing so is its language integrated query (LINQ) suite, which transfers specialized, elegantly-written code into an efficient SQL query.

Beside standard libraries, state-of-the-art structual engineering utilities will likely be used to aid calculations, for implementing complex calculations is prone to error, especially for developers not well-versed in civil engineering. A likely option is the AutoCAD application programming interface (API) with the Inventor API of AutoDesk Inventor being an alternate.

Linting shall be used to ensure production code is free of syntax errors, style issues and bad practices. One option is CS Lint by Lucas Pimentel, freely available as it is licensed under the MIT license.

A unit testing framework shall be implemented to ensure correctness, like the native MS Test, NUnit or xUnit. The comprehensiveness of the unit tests shall be measured in part by code coverage, like the Coverlet tool of XUnit. Another tactic to increase comprehensiveness could be to correlate the projected frequency of execution of a path with the number of unit tests it has. Such frequency would be determined through consultation with this project's users.

Continious integration shall be implemented through a distributed version-control system, specifically git on a GitHub-hosted repository. Each change to production shall be proposed by its developer through a merge request from a feature branch which is exclusively theirs to the production branch. Barring an extraordinary emergency, the change must be approved by another developer before merging, preferably one who is working on a feature which closely interacts with the proposed change.

12 Use of AI and LLMs

This project is not a machine learning project, as it will implement many determinate services for the Troitsky club. However, utilization of robust AI chatbots (e.g. Claude Code, ChatGPT, Copilot) to write code will increase developer efficiency by decreasing the time each developer needs to spend drafting and revising code. However, AI is prone to hallucinations and other vulnerabilities. Therefore, each developer shall throughly review all AI-generated code they generate before creating a pull request and not submit the code for a pull request until they completely understand the code and are convinced of its validity.

13 Coding Standard

This project shall adopt the standard .NET coding conventions, available here.

LINQ shall be utilized unless the equivalent raw SQL query provides a significant performance boost.

14 Appendix — Generative AI Use Disclosure

14.1 AI Tools Used

No AI was used in the creation of this document, except for the spell-checking and grammar services built into the Visual Studio Code editor. Some common free models were used to guide research into tool choice and development plan choices, specifically the core libraries and frameworks adopted for the project.

14.2 How and Where Used

N/A

14.3 Verification of AI-Generated Content

N/A

Appendix — Reflection

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Mason Azzopardi

1. Creating a develpoment plan before starting a project helps outline the expectations of how and what work needs to get done. Additionally a development plan outlines how the team should interact with each other, how eften we meet, where and when we talk about what, how we share information, what everyones general responsibility is. In a sentence a development plan ensures the team is on the same page with regards to how the project should be developed for the duration we are working on it.
2. In my opinion the main advandate of a CI/CD pipeline is to add an extra layer of ensurance on correctness when deploying code to the main product. The main disadvantage is the time it takes to set up CI/CD and that it's not a cumulative error catcher but it gives the confidence that your code is correct when you deploy it.
3. We had some disagreements over how to manage attaching issues to branches, for this deliverable I thought it would be fine for all of us to just have our

work on a singular branch each and then merge at the end, however the rest of the group argued that we should do it based on each issue and to be honest they were just right. So we went with that.

Pedro Azevedo

1. The creation of the development plan prior to starting the project is key to establishing the patterns and strategies that the team will use to create clean and maintainable code. Part of the value of this project is its ability to stand without anyone from the current group being a part of it, and central to that will be having well-documented code. The development plan also made us settle on some key factors of the project that may have been decided more on a whim if it wasn't set in stone.

2. I believe the main benefit of CI/CD is the standardization of builds and submissions, ensuring that what we work on will be properly replicated when built by anyone else, including the TAs and instructors. It ensures that the main branch is always a working version of the service, one that has passed all the checks set up by the team. It brings peace of mind when the project gets busier, and development is moving quicker , ensuring no changes that break the project are made.

3. There weren't any major disagreements during this deliverable, just different suggestions for technologies for the development plan, which were all decided through some more thorough research. When certain decisions relied on some technical knowledge, the team would often turn to the member who best understood that topic.

Jack Easton

1. During writing the this deliverable, it really gave our team a great opportunity to come together and unify our perpsectives and ideas. All team members had many invaluable ideas and inputs on the project which allowed us to end up with a proposed development plan and problem statement that is greater than what we could have come up with individually.

2. Intially when writing the content and documentation for this deliverable, it was difficult to focus all of the different ideas, options, tools, and pieces of domain knowledge provided by the Troitsky Team into a clear and articulatable form. However, through working through proven processed for development and figuring things out one question/bulletpoint at a time, I feel much more confident in my understanding and ability to convey our plan for the POC document.

3. I think the key of the scope of our project being suitable for a Capstone project lies in the unique structure of our project. Because concept of the project is to create a toolbox of a variety of independent but specially

customized tools, each tool presents isolated complexity. This naturally allows the project to be broken down into attainable sections (by tool). When viewed individually, some of the tools are highly complex (making them suitable for a senior design project), but all of the tools do not relpy on each other to present their functionality. It is additionally worth noting that a large part of the complexity of the project comes in how to correctly and efficently present a multitude of differing features while adhering to proper software architecture design principles.

Michael Roberts

1. In one adage, whoever fails to plan plans to fail. This design document includes the current inputs to our project, the expected outputs of our projects, plainly states knowledge that otherwise would be implicit, like what information is confidential and who our stakeholders are. Lastly, formalizing our workflow is critical to minimize future disagreements about it, and establishing a team code of conduct is important to establish a predictable response to unpredictable situations that may occur in the future.

2. In my opinion, the main benefit of CI/CD is that the main branch safely and frequently updates to reflect new features and bug fixes without losing developer synchronization. This avoids manual resolution of large merge conflicts and through standard linting, stylistic and programming errors are automatically caught, as well as code that fails various automated semantic tests defined by the developers. The great challenge is dividing the tasks so that they are small, or segregating the necessary large task away from the rest of the system to avoid the inevitable merge conflict. Another one is the the break glass scenario, where the tests and peer review must be overriden to fix a critical error, which can wreak havoc later.

3. There were not any major disagreements. When we would disagree, we would respectfully discuss and voice our opinions and amicably come to a compromise or one party would concede after discovering their view is inferior through an open mind during discussion. Deferring to the most knowledgeable member was a critical tool and will serve me well as I become a software engineer.

Appendix — Team Charter

External Goals

1. The primary external goals related to the benefit and success of this product in improving McMaster's Troitsky teams; fundamentally, we want to have a positive impact on the club as a whole. This project seeks to bring some legitimacy to the club's operations, improving the general quality of life of members and hopefully attracting more interest in the club, both from normal members and, additionally interested software members.

2. Another external goal would be to have a highly organized and streamlined development process throughout the entire project. With the use of all the GitHub features, the team will lock into an iterative development where all members' input and feedback can be properly assessed and organized to further document our work. The team wants this project to showcase our abilities to collaborate and develop a large-scale project smoothly.

Attendance

The section describes the rules and expectations outlined for team member attendance.

Expectations

Our team expects near-full attendance at all scheduled meetings, along with the required work for the meetings done beforehand. Members should be on time to scheduled meetings, and justification for leaving early must be discussed with the other members in advance. While full attendance is the standard, there will be room for members to make notice that they will be absent at a future meeting, with the added task of supplying required information before the meeting and catching up on meeting notes after. If a member misses 3 pre-scheduled meetings, there must be strong justification, and an additional meeting will be scheduled with the other members to accommodate and discuss. Approaching deliverables, meeting attendance becomes more vital and therefore after a single missed scheduled meeting, absences must be replaced with a supporting meeting or direct discussion with a member about the meeting contents. For the scheduled meetings, in-person attendance is generally expected, but there will be room for virtual attendance when other members are notified beforehand.

Acceptable Excuse

For internal deadlines that the team has set, the expectation is that other members are kept on track of the progression of work and therefore made aware that a deadline may be missed before it passes. This will ensure that if necessary, other members may assist in the required work or those internal deadlines may be adjusted. For missing meetings or when missed deadlines must be justified,

acceptable excuses will range from standard academic regulations: sickness, family responsibilities, religious accommodations, and any previously communicated commitments that the team has been made aware of. For real project deadlines, there will be no acceptable excuses unless any true emergencies have occurred.

In Case of Emergency

Support for emergency situations will begin with the continuous documentation of ongoing work, allowing for simplified transfer of assignment of tasks. If an emergency occurs and a team member either will not be able to complete their work or is even unsure of their ability to do so, they must contact and inform the team as soon as possible. The rest of the team will then attempt to accommodate and support the member by redistributing their workload, ensuring the completion of work that is to the team's standard.

Accountability and Teamwork

Quality

The team will expect nothing less than the best effort of each member, seeking to create a final product that everyone is proud to be a part of. Each member is looking to not only excel in the class, but also function smoothly as a team in the process, so preparation for meetings will be vital. Just like the final product, this preparation must be held to a high standard, ensuring that complete team progress is made smoothly.

Expectations of effort may be discussed on a per-meeting basis, ensuring that all members are on the same page about the work required to achieve the desired final product. As the scope for this project is adjustable, this communication between members will be central to keeping expectations in check.

Attitude

Central to the team's success will be remaining respectful, understanding and focusing on collaboration amongst one another. A positive attitude is generally expected, but as with any project, members may become stressed, and in those situations respect must be maintained, and discussion will be had if something breaks this expectation. Each member will also be expected to contribute a fair share of the work, both within the technical development of the project and the external tasks required. Conflict resolution will be done in a calm and respectful manner, focusing on maintaining an environment that, even through conflict, remains in check. Meetings may be scheduled with the TA and/or professor if issues escalate beyond what is feasible for the team to handle.

Stay on Track

Staying on track will be rooted in organization and communication among members, primarily utilizing GitHub task assignment features. Beyond that, team members often attend classes together, and capstone progress is always discussed, so there is a constant general understanding of how the project develops. The weekly meetings will then formalize these updates, ensuring that all progress is documented. Team and community interactions are guided by the code of conduct outlined in the project folder, ensuring that a respectful and considerate environment is always maintained. Expectations of members' standards as discussed above have all been a general priority of all the individual members even prior to forming this team, so adherence to these expectations is rooted in the group's interest. If a member does happen to miss a meeting or an internal deadline, a coffee or treat should be bought for the other members during the next meeting. For more important formal deadlines, any issues that arise from absences or missed submissions will result in discussion with the TA and/or professor.

Team Building

The team will build cohesion through the attendance of Troitsky events and meetings, as the team's members are introduced as formal members of the club. These events will provide a more casual place to build a team bond, while also allowing everyone the opportunity to engage with the eventual users of the platform, gaining a deeper understanding of the club itself.

Decision Making

Decisions in the group will be made democratically through a vote, with decisions involving several parameters allowing for a ranking of voting options. All split decision handling will begin with more discussion of the topic, where those with more opinion on the subject may state their reasoning and appeal to the others. In the case where it remains a tied vote, the supervisor will be consulted and will ultimately decide.